

Projet B - Séquence 4.4 - Croisement de cohérence

Voici une **analyse de cohérence complète** du modèle de données proposé par rapport aux **fonctionnalités MVP**, avec identification des **manques**, **incohérences**, **risques de duplication**, et **relations incorrectes**.

1. Croisement Fonctionnalités MVP ↔ Modèle de Données

Pour chaque fonctionnalité MVP, vérifions si elle est **réalisable** avec le modèle actuel.

Fonctionnalité 1 : Créer un compte et se connecter

Besoin :

- Stockage des utilisateurs (étudiants/professeurs) avec authentification.
- Gestion des rôles.

Modèle associé :

- **Table Utilisateur :**
 - Champs : `id`, `nom`, `prénom`, `email`, `mot_de_passe`, `rôle`, `date_inscription`, `statut_compte`, `dernière_connexion`.
 - **OK** : Tous les champs nécessaires sont présents.
 - **Amélioration** :
 - Ajouter un champ `email_verifié` (BOOLEAN, défaut `false`) pour valider l'email avant connexion.
 - Ajouter un champ `jeton_reinitialisation` (Text, `NULL`) pour la réinitialisation du mot de passe.

Verdict :  **Réalisable**, mais manque `email_verifié` et `jeton_reinitialisation` pour une authentification robuste.

Fonctionnalité 2 : Créer et personnaliser son profil

Besoin :

- Personnalisation du profil (nom, prénom, bio, photo, etc.).

Modèle associé :

- Table **Utilisateur** :
 - Champs : nom, prénom, bio, photo_profil.
 - **OK** : Tous les champs sont présents.

Verdict :  **Réalisable sans modification.**

 Fonctionnalité 3 : Visualiser un calendrier intégré

Besoin :

- Chaque utilisateur a un calendrier personnel avec des événements (tâches, échéances, etc.).

Modèle associé :

- Tables **Calendrier** et **Événement** :
 - **Calendrier** : id, utilisateur_id (1-à-1).
 - **Événement** : id, titre, description, date_debut, date_fin, calendrier_id, couleur.
 - **Problème** :
 - Les **tâches** et **échéances** ne sont pas automatiquement liées au calendrier.
 - **Manque** : Un mécanisme pour **synchroniser les tâches/échéances avec le calendrier** (ex : champ **calendrier_id** dans **Tâche** ou **Échéance**, ou un déclencheur pour créer un **Événement** à partir d'une **Tâche**).

Risque :

- **Duplication de données** : Si une tâche a une **date_limite**, cette information doit aussi apparaître dans le calendrier sous forme d'**Événement**.
- **Incohérence** : Si une tâche est modifiée, son événement associé dans le calendrier ne sera pas mis à jour automatiquement.

Solution proposée :

- Ajouter un champ **calendrier_id** dans **Tâche** et **Échéance** (optionnel) pour lier directement à un calendrier.
- **OU** : Créer un **déclencheur (trigger)** en base de données pour :

- Ajouter un **Événement** dans **Calendrier** quand une **Tâche** ou **Échéance** est créée/modifiée.
- Supprimer l'**Événement** si la **Tâche/Échéance** est supprimée.

Verdict : ⚠ **Réalisable mais risque d'incohérence** sans synchronisation automatique.

✅ **Fonctionnalité 4 : Ajouter, modifier et supprimer des tâches**

Besoin :

- Gestion complète des tâches (CRUD), liées à un cours ou un groupe.

Modèle associé :

- **Table **Tâche** :**
 - Champs : **id**, **titre**, **description**, **date_creation**, **date_limite**, **statut**, **priorité**, **cours_id**, **groupe_id**, **utilisateur_id**.
 - **OK** : Tous les champs nécessaires sont présents.
 - **Problème** :
 - **Cardinalité** : Une tâche peut être liée à **un cours OU un groupe**, mais pas aux deux en même temps (champs **cours_id** et **groupe_id** sont optionnels).
 - **Risque** : Si une tâche est liée à un cours **et** un groupe, cela peut créer une ambiguïté (ex : à quel contexte appartient-elle vraiment ?).
 - **Solution** :
 - **Soit** : Rendre **cours_id** et **groupe_id** **mutuellement exclusifs** (via une contrainte CHECK en SQL).
 - **Soit** : Créer une **table de jonction **Tâche_Contexte**** pour lier une tâche à plusieurs cours/groupes (si besoin futur).

Verdict : ✅ **Réalisable**, mais à clarifier pour éviter les ambiguïtés.

✅ **Fonctionnalité 5 : Accéder à un tableau de bord principal**

Besoin :

- Vue d'ensemble des tâches, échéances, notifications, et progrès.

Modèle associé :

- **Tables impliquées** :
 - **Tâche** (statut, priorité, date_limite).
 - **Échéance** (date, description).
 - **Notification** (type, lu, date_envoi).

- **Progression** (note, date_realisation).
- **OK** : Toutes les données nécessaires sont présentes.
- **Manque** :
 - **Aucun champ pour le "tableau de bord"** : Il manque une **vue ou une table dédiée** pour agréger ces données (ex : **TableauDeBord** avec des métriques comme "nombre de tâches en retard", "notes moyennes", etc.).
 - **Solution** :
 - **Créer une vue SQL** (ex : **vue_tableau_de_bord**) qui agrège :
 - Nombre de tâches par statut (à **faire**, **en cours**, **terminée**).
 - Nombre de notifications non lues.
 - Dernières échéances à venir.
 - Moyenne des notes (via **Progression**).

Verdict : ⚠ **Réalisable mais nécessite une vue SQL** pour le tableau de bord.

✅ **Fonctionnalité 6 : Créer et rejoindre des groupes collaboratifs**

Besoin :

- Création de groupes par les utilisateurs (étudiants/professeurs).
- Gestion des membres (rôles : admin/membre, validation d'adhésion).

Modèle associé :

- **Tables **Groupe** et **Utilisateur_Groupe**** :
 - **Groupe** : id, nom, description, date_creation, créateur_id, statut.
 - **Utilisateur_Groupe** : id, utilisateur_id, groupe_id, rôle, date_adhesion, statut.
 - **OK** : Tous les champs nécessaires sont présents.
 - **Problème** :
 - **Cardinalité** : Un groupe peut avoir **plusieurs admins** (si un admin ajoute un autre admin).
 - **Risque : Incohérence** si le créateur du groupe (**créateur_id** dans **Groupe**) est supprimé de **Utilisateur_Groupe**.
 - **Solution** :
 - **Ajouter une contrainte** : Le **créateur_id** doit toujours avoir un enregistrement dans **Utilisateur_Groupe** avec **rôle = 'admin'**.
 - **Empêcher la suppression** du créateur via un trigger.

Verdict : ✅ **Réalisable**, mais nécessite des contraintes pour éviter les incohérences.

✓ Fonctionnalité 7 : Accéder à des ressources personnalisées (quiz, fiches, vidéos)

Besoin :

- Stockage et accès aux ressources (créées par étudiants/professeurs).
- Catégorisation des ressources.

Modèle associé :

- Tables **Ressource**, **Catégorie**, **Ressource_Catégorie** :
 - **Ressource** : `id`, `titre`, `type`, `contenu`, `date_creation`, `utilisateur_id`, `visibilité`.
 - **Catégorie** : `id`, `nom`, `couleur`.
 - **Ressource_Catégorie** : `id`, `ressource_id`, `catégorie_id`.
 - **OK** : Tous les champs nécessaires sont présents.
 - **Problème** :
 - **Champ `contenu`** : Stocke à la fois du texte et des URLs. **Risque de confusion** si le type de ressource change (ex : une vidéo doit avoir une URL, une fiche du texte).
 - **Solution** :
 - **Séparer en deux champs** :
 - `contenu_texte` (TEXT, NULL).
 - `contenu_url` (TEXT, NULL).
 - **Ajouter une contrainte** : Si `type` = 'vidéo', alors `contenu_url` doit être non nul.

Verdict : ⚠ Réalisable mais risque de confusion dans le champ `contenu`.

✓ Fonctionnalité 8 : Recevoir des notifications pour les échéances et rappels

Besoin :

- Notifications liées aux tâches, échéances, ou messages.

Modèle associé :

- Table **Notification** :
 - Champs : `id`, `titre`, `contenu`, `date_envoi`, `lu`, `utilisateur_id`, `tâche_id`, `échéance_id`, `message_id`, `type`.
 - **OK** : Tous les champs nécessaires sont présents.
 - **Problème** :
 - **Cardinalité** : Une notification peut être liée à **une tâche OU une échéance OU un message**, mais pas à plusieurs en même temps.

- **Risque** : Si une notification est liée à une **Tâche** qui a plusieurs **Échéance**, il faudra créer **une notification par échéance**.
- **Solution** :
 - **Accepter le modèle actuel** (1 notification = 1 déclencheur).
 - **OU** : Créer une **table de jonction Notification_Contexte** pour lier une notification à plusieurs contextes (ex : une tâche + ses échéances).

Verdict :  **Réalisable**, mais peut générer beaucoup de notifications redondantes.



2. Données Manquantes

Voici les **champs ou tables manquantes** pour couvrir toutes les fonctionnalités MVP **sans risque** :

Fonctionnalité	Donnée manquante	Solution proposée	Priorité
Authentification	email_verifié (BOOLEAN)	Ajouter dans Utilisateur .	★★★★
Authentification	jeton_reinitialisation (Text)	Ajouter dans Utilisateur .	★★★★
Calendrier	Synchronisation Tâche/Échéance → Événement	Ajouter calendrier_id dans Tâche/Échéance OU créer un trigger.	★★★★
Tableau de bord	Vue agrégée des métriques	Créer une vue SQL vue_tableau_de_bord .	★★★
Ressources	Séparation contenu_texte/contenu_url	Remplacer contenu par deux champs.	★★★
Groupes	Contrainte pour le créateur	Trigger pour empêcher la suppression du créateur dans Utilisateur_Groupe .	★★★



3. Relations Incorrectes ou Cardinalités Mal Définies

Relation	Problème	Cardinalité actuelle	Cardinalité correcte	Solution
----------	----------	----------------------	----------------------	----------

Utilisateur ↔ Cours	Un étudiant peut participer à plusieurs cours, un professeur peut donner plusieurs cours.	1-à-N (professeur → cours)	N-à-N (via une table de jonction)	Créer une table Utilisateur_Cours avec <code>utilisateur_id</code> , <code>cours_id</code> , et <code>rôle</code> (étudiant/professeur).
Tâche ↔ Cours/Groupe	Une tâche peut être liée à un cours OU un groupe, mais pas les deux.	1-à-N (optionnel)	1-à-1 (exclusif)	Ajouter une contrainte CHECK : <code>cours_id IS NULL OR groupe_id IS NULL</code> .
Groupe ↔ Utilisateur	Un groupe a un créateur (<code>créateur_id</code>), mais celui-ci peut être supprimé de Utilisateur_Groupe .	1-à-N	1-à-N avec contrainte	Trigger pour empêcher la suppression du créateur.
Notification ↔ Tâche/Échéance/Messagerie	Une notification peut être liée à un seul type de contexte.	1-à-1 (optionnel)	1-à-1 (exclusif)	Ajouter une contrainte CHECK : <code>tâche_id IS NOT NULL OR échéance_id IS NOT NULL OR message_id IS NOT NULL</code> .

! 4. Risques de Duplication ou d'Incohérence

Risque	Cause	Impact	Solution
Duplication des dates	<code>Tâche.date_limite</code> et <code>Événement.date_debut/date_fin</code> peuvent contenir la même information.	Incohérence si l'un est modifié sans l'autre.	Trigger pour synchroniser <code>Tâche</code> → <code>Événement</code> .
Incohérence des groupes	Le créateur d'un groupe (<code>créateur_id</code>) peut être supprimé de Utilisateur_Groupe .	Perte de l'admin du groupe.	Trigger pour bloquer la suppression.

Ressources mal typées	<code>Ressource.contenu</code> mélange texte et URLs.	Difficile à exploiter.	Séparer en <code>contenu_texte</code> et <code>contenu_url</code> .
Notifications redondantes	Une tâche avec plusieurs échéances génère plusieurs notifications.	Spam pour l'utilisateur.	Regrouper les notifications (ex : 1 notification = "Vous avez 3 échéances demain").

5. Modèle Corrigé et Optimisé

Voici les **modifications critiques** à apporter pour garantir la cohérence :

♦ Tables à Ajouter

1. **Utilisateur_Cours** (Table de jonction pour N-à-N entre Utilisateur et Cours)

Champ	Type	Obligatoire	Description
<code>id</code>	UUID	✓	Identifiant unique
<code>utilisateur_id</code>	UUID (FK)	✓	Utilisateur (étudiant ou professeur)
<code>cours_id</code>	UUID (FK)	✓	Cours associé
<code>rôle</code>	ENUM('étudiant', 'professeur')	✓	Rôle dans le cours

2.

3. **Vue SQL** `vue_tableau_de_bord` (Pour agréger les données du tableau de bord)

♦ Champs à Modifier

Table	Champ	Modification	Raison
-------	-------	--------------	--------

Utilisateur	Ajouter <code>email_vérifié</code>	BOOLEAN, défaut <code>false</code>	Pour la validation des emails.
Utilisateur	Ajouter <code>jeton_reinitialisation</code>	Text, NULL	Pour la réinitialisation du mot de passe.
Ressource	Remplacer <code>contenu</code>	Par <code>contenu_texte</code> (TEXT) et <code>contenu_url</code> (TEXT)	Éviter la confusion.
Tâche	Ajouter contrainte	CHECK (<code>cours_id IS NULL OR groupe_id IS NULL</code>)	Éviter les ambiguïtés.
Notification	Ajouter contrainte	CHECK (<code>tâche_id IS NOT NULL OR échéance_id IS NOT NULL OR message_id IS NOT NULL</code>)	Garantir qu'une notification a un contexte.

♦ Contraintes à Ajouter

1. Trigger pour synchroniser **Tâche** et **Événement** :
2. Trigger pour protéger le créateur du groupe :

✅ 6. Validation Finale du Modèle

Avec ces corrections, le modèle : **✅ Couvre toutes les fonctionnalités MVP** sans manque. **✅ Évite les duplications** (via triggers et contraintes). **✅ Gère les cardinalités correctement** (N-à-N pour `Utilisateur_Cours`, 1-à-N exclusif pour `Tâche`). **✅ Préviend les incohérences** (synchronisation calendrier/tâches, protection du créateur de groupe).

Prochaine étape :

- Veux-tu que je génère le schéma SQL complet (avec tables, contraintes, triggers, et vue) pour Supabase/PostgreSQL ?

- **Ou préfères-tu discuter d'autres ajustements avant ?**